

## **Análisis y Diseño de Software**

Nombre:

José Ángel Casiani Echeverría 6° Semestre

Unicolombo

2410180074

[jose.casianiecheverria@unicolombo.edu.co](mailto:jose.casianiecheverria@unicolombo.edu.co)

**José Eduardo Blanco Taboada**

**Unicolombo**

[jose.blancotaboada@unicolombo.edu.co](mailto:jose.blancotaboada@unicolombo.edu.co)

Profesor:

Jonathan Tena

# Explicación integral del proyecto

## AutoDrive Motors

*Documento técnico-académico elaborado a partir del código fuente, el esquema SQL y la documentación del repositorio.*

### Contenido

- 1. Resumen ejecutivo y contexto
- 2. Objetivos y alcance
- 3. Arquitectura y carpetas
- 4. Modelo de datos
- 5. Módulos y procesos
- 6. API REST
- 7. Reglas de negocio
- 8. Tasa de cambio
- 9. Interfaz web
- 10. Validación y errores
- 11. Configuración y arranque
- 12. Pruebas
- 13. Límites y conclusión

# 1. Resumen ejecutivo y contexto

AutoDrive Motors es una aplicación web académica para administrar clientes, inventario de vehículos, ventas y servicios asociados a los vehículos. El backend publica una API REST y la misma aplicación Spring Boot sirve la interfaz web.

Aborda dificultades del registro manual: duplicidad de placas o correos, precios invalidos, incertidumbre sobre la disponibilidad, historial disperso y demora en consultar operaciones. El principio del diseño es que el estado visible del inventario coincida con las transacciones guardadas.

## Objetivo del sistema

Centralizar las operaciones de una concesionaria en una plataforma web y aplicar validaciones tanto en la capa de negocio como en la base relacional. Las ventas se conservan como historial y los errores se explican al usuario.

## 2. Objetivos y alcance

### Objetivos específicos

- Gestionar clientes y exigir correo único.
- Registrar vehículos, filtrarlos por marca y consultar disponibilidad.
- Calcular en servidor fecha, descuento y total de venta.
- Evitar vender vehículos no disponibles y prevenir duplicidad bajo concurrencia.
- Registrar servicios e historial; actualizar el estado del vehículo según los servicios abiertos.
- Mostrar conversiones USD orientativas y no perder la venta si falla la tasa externa.
- Devolver errores con códigos HTTP y mensajes claros.

### Actores y fronteras

El usuario principal es el personal de la concesionaria. PostgreSQL conserva datos; un servicio remoto informa la tasa USD/COP. En el repositorio no se documentan autenticación/autorización ni despliegue productivo automatizado. No se permite borrar ventas, pues forman parte del historial comercial.

### 3. Arquitectura y organización de carpetas

La aplicación sigue una arquitectura por capas: el Controller recibe HTTP; DTO define contratos; Service ejecuta reglas y transacciones; Repository consulta y persiste; Entity modela el dominio; Mapper convierte entidades a respuestas. TasaCambioClient encapsula la consulta externa y GlobalExceptionHandler estandariza errores.

#### Responsabilidad de los paquetes

- controller/: rutas, métodos HTTP y delegación.
- dto/request y dto/response: contratos de solicitud y respuesta, separados de entidades.
- service/: reglas de negocio, calculos y transacciones.
- repository/: persistencia y consultas con Spring Data JPA.
- entity/: entidades y enumeraciones persistidas.
- mapper/: conversión de Entity a DTO.
- client/: integración HTTP de tasa de cambio.
- exception/: errores de dominio y manejador global.
- config/: carga de datos iniciales, desactivable por propiedad.

#### Árbol real del repositorio

```
TallerAutos/  
|-- src/main/java/com/tallerautos/  
|   |-- TallerAutosApplication.java  
|   |-- client/ config/ controller/  
|   |-- dto/request/ dto/response/  
|   |-- entity/ exception/ mapper/  
|   |-- repository/ service/  
|-- src/main/resources/  
|   |-- application.properties  
|   |-- static/index.html  
|   |-- static/css/estilos.css  
|   |-- static/js/app.js  
|   |-- static/img/ y fuentes/  
|-- src/test/java/com/tallerautos/  
|   |-- ContextoApplicationTest.java  
|   |-- service/VentaServiceTest.java  
|   |-- service/Rnf008DegradacionTest.java  
|-- Diagramas/ UML, casos de uso, secuencias y DER  
|-- schema.sql reset_datos.sql reset_vehiculos.sql  
|-- pom.xml README.md PRODUCT.md Postman_*.json
```

El frontend es estático y se sirve desde Spring Boot; no hay build frontend independiente. Diagramas contiene artefactos de análisis. README describe el arranque; Postman y los SQL de reset apoyan demostraciones y verificaciones.

## 4. Modelo de datos y relaciones

schema.sql define cuatro tablas, con ids generados, claves foraneas y restricciones. En el entorno normal Hibernate utiliza ddl-auto=validate: verifica que las entidades correspondan al esquema, pero no lo crea ni lo modifica.

### Entidades

- Cliente: nombre, email, telefono, direccion y fecha de registro. El correo es único.
- Vehículo: placa, marca, modelo, año, color, precio, estado y fecha de ingreso. La placa es única.
- Venta: referencias a cliente y vehículo, fecha, valor, descuento, total, tasa y equivalente USD.
- Mantenimiento: vehículo, tipo, descripción, costo, fechas de inicio/fin y estado.

### Relaciones y restricciones

- Cliente 1:N Venta: cada venta pertenece a un cliente.
- Vehículo 1:N Mantenimiento: conserva el historial por unidad, del más reciente al más antiguo.
- La tabla Venta permite referenciar un vehículo, mientras RN-1 limita la venta efectiva a una por unidad con bloqueo en servicio.
- CHECK limita estados permitidos y montos no negativos; UNIQUE protege email y placa.
- Índices soportan consultas por estado/marca, ventas por cliente/vehículo e historial de servicio.

## 5. Módulos y procesos principales

### Clientes

ClienteService recorta texto, normaliza el correo a minúsculas y comprueba duplicados en alta y actualización. No permite eliminar clientes que ya tengan ventas, para proteger el historial.

### Vehículos

VehiculoService normaliza placa a mayúsculas, verifica unicidad y ofrece listados, búsqueda por id, marca y disponibilidad. No elimina unidades con ventas o servicios asociados. La conversión USD en consultas es informativa.

### Ventas

VentaRequest contiene los ids de cliente y vehículo. El servidor recupera las entidades, bloquea la fila del vehículo, válida DISPONIBLE, calcula descuento y total, consulta tasa real, guarda la venta y cambia estado a VENDIDO dentro de una transacción.

### Servicios

Crear un servicio lo inicia EN\_PROCESO y cambia el vehículo a EN\_MANTENIMIENTO. El costo y descripción se pueden editar mientras siga abierto. Al cerrar, el vehículo vuelve a DISPONIBLE solo si no quedan servicios abiertos; no se abre servicio para un vehículo vendido.

### Datos iniciales

DatosIniciales ejecuta al inicio y puede desactivarse con `datos.iniciales.activar=false`. Comprueba tablas antes de poblar ejemplos para facilitar una demostración sin duplicar datos completos ya existentes.

## 6. API REST: 23 operaciones

Todas las rutas de negocio comienzan con /api. Los controladores agrupan 5 recursos: clientes, vehículos, ventas, mantenimientos y tasa-cambio.

```
CLIENTES
GET /api/clientes | POST /api/clientes
GET /api/clientes/{id} | PUT /api/clientes/{id} | DELETE /api/clientes/{id}
VEHÍCULOS
GET /api/vehículos | POST /api/vehículos
GET /api/vehículos/{id} | PUT /api/vehículos/{id} | DELETE /api/vehículos/{id}
GET /api/vehículos/disponibles | GET /api/vehículos/marca/{marca}
VENTAS
POST /api/ventas | GET /api/ventas
GET /api/ventas/cliente/{clientId} | GET /api/ventas/vehículo/{vehiculoid}
GET /api/ventas/simular/{vehiculoid}
MANTENIMIENTOS
POST /api/mantenimientos | GET /api/mantenimientos
GET /api/mantenimientos/vehículo/{vehiculoid}
PUT /api/mantenimientos/{id} | PATCH /api/mantenimientos/{id}/cerrar
TASA
GET /api/tasa-cambio
```

Los POST de creación devuelven 201 Created; el borrado válido responde 204 No Content. La simulación permite ver total y descuento antes de confirmar. No existe DELETE para ventas: una venta no se elimina.

## 7. Reglas de negocio

- RN-1: solo se vende un vehículo DISPONIBLE. VentaService toma bloqueo pesimista de escritura antes de validar para evitar doble venta concurrente.
- RN-2: no se permiten precios negativos; se comprueban en DTO y con CHECK SQL.
- RN-3: la placa se normaliza y debe ser única en servicio y base.
- RN-4: el correo se normaliza y debe ser único en servicio y base.
- RN-5: fecha y total no son datos confiados al cliente; el servidor los calcula.
- RN-6: descuento del 5% solo si el precio supera estrictamente \$100.000.000 COP. En el valor exacto no aplica.
- Un vehículo permanece EN\_MANTENIMIENTO mientras tenga un servicio abierto; al cerrar el último queda DISPONIBLE.
- Se rechaza el borrado de clientes/vehículos que tienen historial asociado.

### Importes y concurrencia

Los importes usan BigDecimal. El total es  $\text{precio} \times (1 - \text{descuento}/100)$  con redondeo de dos decimales HALF\_UP. compareTo compara magnitudes numericas independientemente de la escala. El bloqueo evita que dos operaciones validen al mismo tiempo el mismo estado disponible.

## 8. Tasa de cambio y tolerancia a fallos

TasaCambioClient consulta [open.er-api.com/v6/latest/USD](https://open.er-api.com/v6/latest/USD) y extrae la tasa COP y su fecha.

TasaCambioService mantiene una tasa válida en cache durante 30 minutos. `application.properties` declara una referencia de 3200 COP por USD.

### Diferencia entre lectura y venta

- Con proveedor disponible: se usa tasa real. La venta guarda tasa y valor USD.
- Si falla: una lectura puede mostrar valor orientativo con tasa de respaldo y comunicar su origen.
- En una venta no se usa la tasa de respaldo para fijar dólares: total COP se conserva y tasa/precio USD quedan nulos.
- El fallo externo no se almacena como tasa correcta y una solicitud posterior puede reintentar.

```
tasa.api.url=https://open.er-api.com/v6/latest/USD
tasa.cache-minutes=30
tasa.valor-por-defecto=3200
tasa.api.timeout-ms=3000
```

## 9. Interfaz web

La interfaz esta en `src/main/resources/static`. Spring Boot entrega la página y la API desde el mismo origen. JavaScript llama rutas relativas `/api`: no requiere otro servidor frontend ni CORS para el uso normal.

- Inicio: indicadores de clientes, vehículos, disponibles, ventas, servicios y tasa.
- Clientes: alta, consulta, edicion, borrado permitido y tabla.
- Vehículos: catalogo, registro, edicion, filtro por marca y solo disponibles.
- Ventas: seleccion de cliente y vehículo disponible, simulacion de valores y confirmacion.
- Servicio: historial, filtro de abiertos, actualización y cierre.
- `app.js` centraliza fetch, formato de moneda/fechas, navegacion por paneles y avisos.
- `estilos.css` da presentacion a formularios, tablas, estados y diseño adaptable.
- Los datos dinamicos se agregan como texto/nodos, evitando interpretar campos como HTML.

## 10. Validación y manejo de errores

Los DTO usan Jakarta Bean Validation; controladores usan @Valid. GlobalExceptionHandler construye ApiError con timestamp, status, mensaje, path y detalles opcionales.

- 200 OK: consulta o actualización completada.
- 201 Created: recurso creado.
- 204 No Content: borrado válido.
- 400 Bad Request: campos de entrada no válidos, con detalle.
- 404 Not Found: no existe el identificador solicitado.
- 409 Conflict: duplicados, regla de negocio o conflicto de integridad.
- 502 Bad Gateway: proveedor externo falla en operaciones que requieren una tasa. La venta trata ese fallo para poder continuar en COP.

# 11. Configuración y puesta en marcha

## Requisitos

- Java 17 y Maven.
- PostgreSQL (la guía del proyecto trabaja con version 18).
- Variable local DB\_PASSWORD para autenticar con la base.

## Inicio

Crear la base tallerautos, ejecutar schema.sql y arrancar Spring Boot. ddl-auto=validate contrasta el modelo persistente con el esquema existente.

```
createdb tallerautos
psql -U postgres -h localhost -d tallerautos -f schema.sql
$env:DB_PASSWORD = "<clave local>"
mvn spring-boot:run
```

La aplicación escucha en el puerto 8080. Abrir <http://localhost:8080/>; abrir el HTML por file:// bloquea las peticiones de navegador.

## Archivos de apoyo

pom.xml define Java 17, Spring Boot 3.3.5, Spring Web, Data JPA, Validation, PostgreSQL y componentes de prueba H2/JUnit. README explica el arranque; reset\_\*.sql limpia datos; los JSON de Postman guardan colección y entorno. Las credenciales se configuran localmente, no se escriben en el repositorio.

## 12. Pruebas y verificación

### JUnit

ContextoApplicationTest arranca el contexto Spring usando H2 en memoria y verifica que los componentes principales se conectan. VentaServiceTest cubre descuento y límite.

Rnf008DegradacionTest verifica tasa real/caída, respaldo solo para lectura y conservación del total en COP.

### Casos importantes

- Debajo de \$100.000.000: sin descuento.
- Exactamente \$100.000.000: sin descuento.
- Un peso por encima: aplica 5%.
- \$120.000.000 con descuento: total \$114.000.000.
- BigDecimal con escalas distintas mantiene comparación numérica correcta.
- Fallo de proveedor no cancela venta y no almacena conversión USD ficticia.

### Postman

La colección Postman organiza peticiones de extremo a extremo y assertions. Debe ejecutarse en orden porque hay pasos que guardan ids y otros que dependen del estado anterior.

## 13. Límites y conclusión

### Aspectos fuera del alcance actual

- No hay autenticación/roles ni despliegue productivo automatizado documentados.
- La tasa de referencia es orientativa y no sirve como valor histórico de venta.
- Se pueden agregar migraciones de esquema, auditoria, observabilidad y reportes paginados.
- Antes de un uso real se deben definir seguridad, permisos, respaldos y operación.

### Conclusión

AutoDrive Motors integra clientes, inventario, ventas y servicios sobre una API REST y una base relacional. Su decisión técnica clave es hacer la venta una transacción protegida por bloqueo, no una acción visual. La separación por capas ubica responsabilidades; las validaciones, restricciones SQL y pruebas ayudan a preservar consistencia. La degradación de tasa externa evita perder operaciones sin congelar valores de cambio inventados.